

Internet of Things – Third Challenge

Project Topics

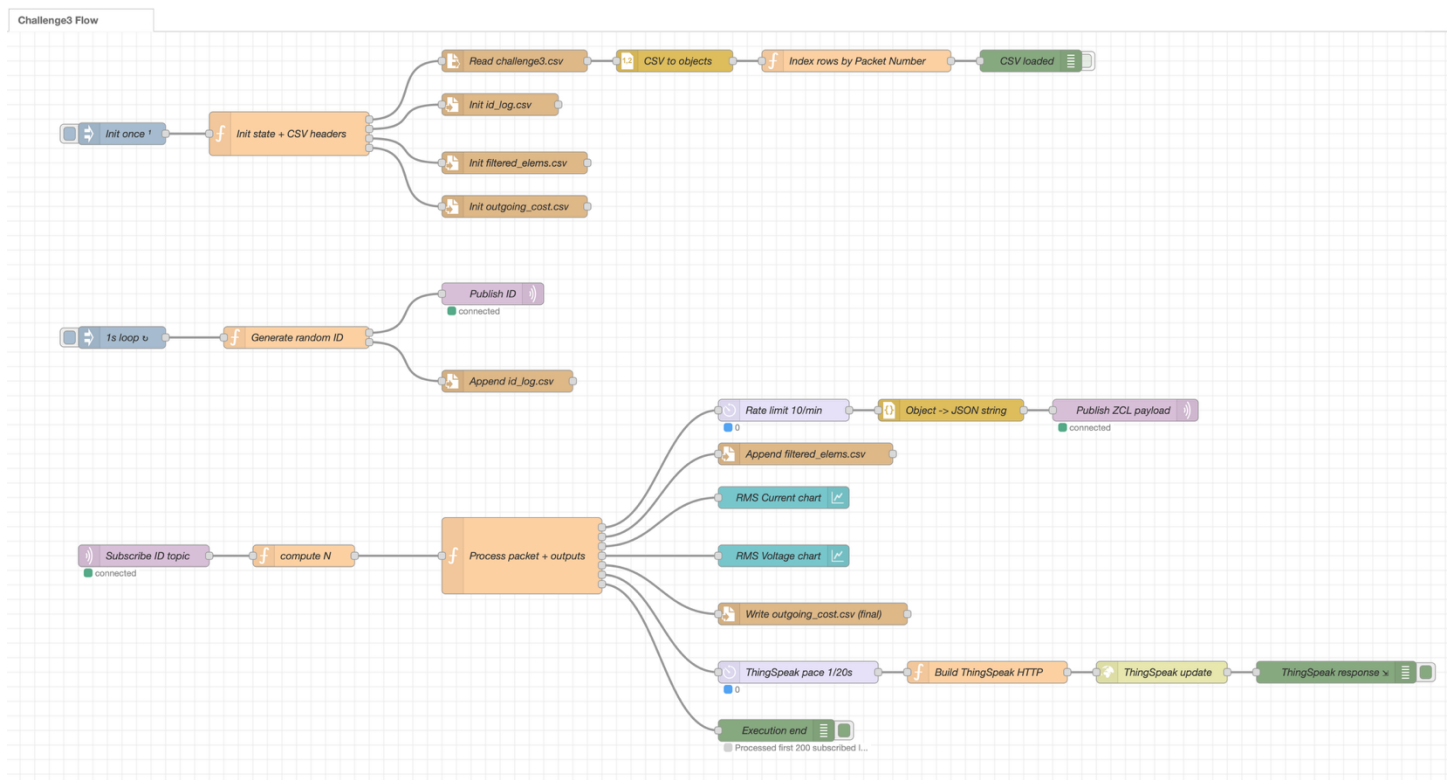
The aim of this challenge was to analyze LoRaWan traffic data using Node-Red flows.

The information is exchanged through a local MQTT server, and the extracted information is visualized on a node-red dashboard and a ThingSpeak channel.

ThingSpeak channel ID: [3368433](#)

GitHub repository: <https://github.com/pitesse/loT/tree/master/challenge3>

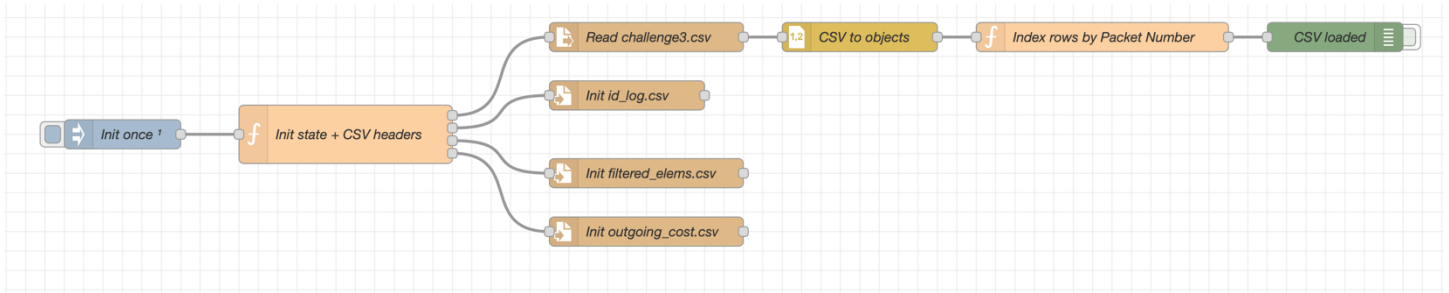
Node-Red flow



The environment is fully dockerized, containing both node red and mosquito, launch through a bash script in the runtime folder inside the challenge3 folder in the repo.

Block review

Init flow



The init block is in charge of initializing the flow variables inside node red to keep track of the number of message ids processed.

it outputs 4 messages: one is a timestamp to trigger reading from the challenge.csv file,
the others are the csv headers to be written to the output csv files.

```
flow.set("processDone", false);
flow.set("subCount", 0);
flow.set("idNo", 0);
flow.set("filteredNo", 0);
flow.set("outgoingCostMap", {});
flow.set("csvIndex", {});

return [
  { payload: Date.now() },
  { payload: "No.,ID,TIMESTAMP\n" },
  { payload: "No.,Timestamp,Sequence Number,Attribute,Status,Data Type,Data Value\n" },
  { payload: "No.,Source,Destination,Cost\n" }
];
```

The indexing block is only needed for debug purposes, it extracts only the packet number and sends it to a debug node to check that all rows have been read from the csv to objects block

```

let rows;

let index = flow.get("csvIndex") || {};

if (Array.isArray(msg.payload)) {
    rows = msg.payload;
    index = {};
} else {
    rows = [msg.payload];
}

for (const row of rows) {
    if (!row) {
        continue;
    }

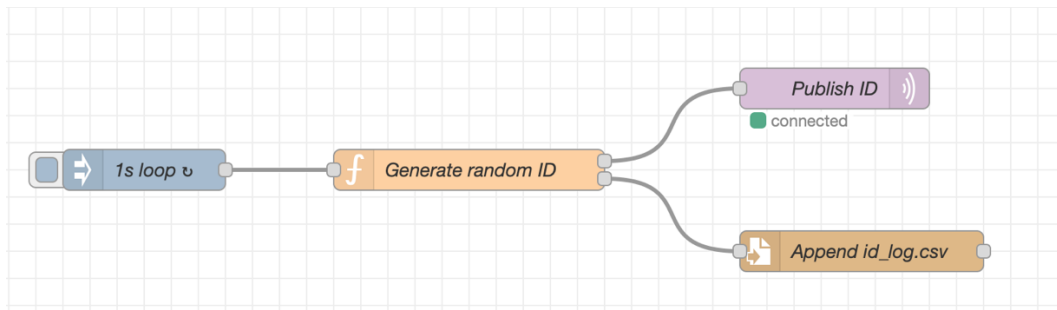
    const n = Number(row["Packet Number"]);
    if (Number.isFinite(n)) {
        index[n] = row;
    }
}

const count = Object.keys(index).length;
flow.set("csvIndex", index);

return { payload: `CSV index ready: ${count} rows` };

```

MQTT publisher flow



This flow manages publishing to the Mosquitto broker on localhost, port 1884, to the topic challenge3/id_generator. The initial inject node ensures messages are sent at a rate of one per second. The code within the Generate Random ID node (function node, on message) is as follows:

```
let no = flow.get("idNo") || 0;

if (no >= 200) {
  return null;
}

no += 1;

flow.set("idNo", no);

const id = Math.floor(Math.random() * 30001);

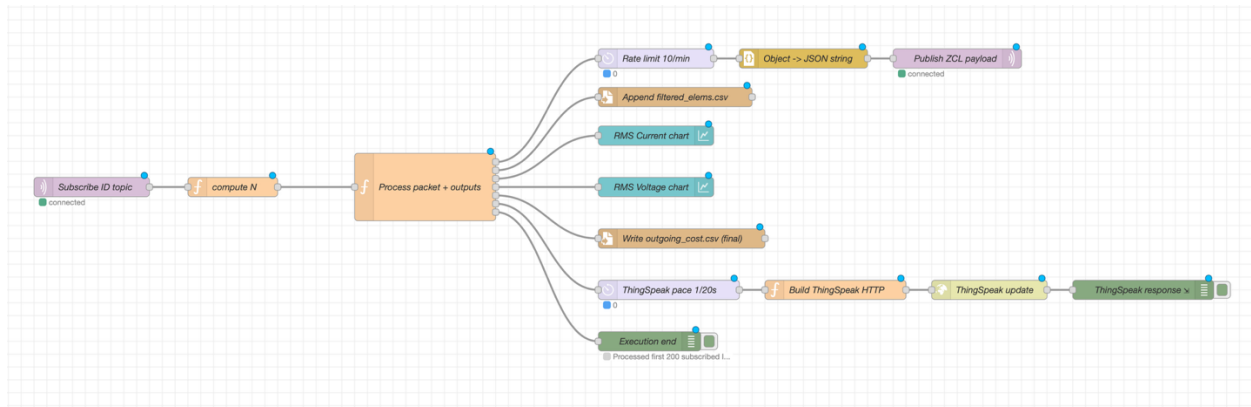
const ts = Math.floor(Date.now() / 1000);

const payload = { id, timestamp: ts };

return [
  { topic: "challenge3/id_generator", payload: JSON.stringify(payload) },
  { payload: `${no},${id},${ts}\n` }
];
```

This code generates the payload for the MQTT broker by creating random IDs and a timestamp. The first output, connected to the MQTT out node, sends this payload. The second output, connected to the write file node, appends the ID logs to the id_log file in the required No.,ID,TIMESTAMP format.

Data extraction flow



This flow is triggered by an mqtt node which subscribes to the challenge3/id_generator topic, the compute N node takes the message id of the notified message, computes N as instructed, and forwards adding it into the payload to the next node.

```
if (flow.get("processDone")) {
    return null;}
let data = msg.payload;
if (typeof data === "string") {
    try {
        data = JSON.parse(data);
    } catch (e) {
        return null;
    }
}
if (typeof data !== "object" || data === null || data.id === undefined) {
    return null;
}
const id = Number(data.id);
if (!Number.isFinite(id)) {
    return null;
}
let count = flow.get("subCount") || 0;
count += 1;
flow.set("subCount", count);
if (count > 200) {
    flow.set("processDone", true);
    return null;
}
if (count === 200) {
    flow.set("processDone", true);
}
const idInt = Math.trunc(id);
msg.sub_id = idInt;
msg.sub_timestamp = Number.isFinite(Number(data.timestamp)) ? Math.trunc(Number(data.timestamp)) : Math.floor(Date.now() / 1000);
msg.sub_count = count;
msg.n = ((idInt % 5218) + 5218) % 5218;
return msg;
```

the process packet node loads the n-th row of the challenge3.csv files, it inspects all the fields to check for current and voltage reading and link status messages.

It extracts and constructs the payload for all its outputs:

Output 1: contains the MQTT message to publish to the topic named after the value in Device Name ZigBee source

Output 2: contains a new filtered_elems.csv row, which is appended to the file

Output 3: contains value for the RMS Current chart

Output 4: contains value for the RMS Voltage chart

Output 5: contains the outgoing_cost.csv row, which is appended to the file

Output 6: contains the ThingSpeak API payload, which is delayed following a message every 20 seconds, then the function node and http request node send the request to publish data on the Thingspeak channel

Output 7: used to send an end message for debug purposes.

```

//Pieces of code are omitted for formatting purposes
[...]
function valueForDataType(dtypeRaw, cmdObj, cursors) {
  const dtypeStr = String(dtypeRaw || "").trim();
  if (!dtypeStr) return "";
  const codeMatch = dtypeStr.match(/\\((0x[0-9a-fA-F]+)\\)/);
  if (!codeMatch) return "";
  const field = DATA_TYPE_TO_FIELD[codeMatch[1].toLowerCase()];
  if (!field) return "";
  const raw = cmdObj[field];
  if (raw === undefined || raw === null) return "";
  const arr = Array.isArray(raw) ? raw.filter(v => v !== undefined && v !== null && v !== "") : [raw];
  if (!arr.length) return "";
  const cursor = cursors[field] || 0;
  cursors[field] = cursor + 1;
  return normalizeValue(arr[cursor] || arr[arr.length - 1] || "");
}

const cmd = parseCommand(packet["Command String"]);
const hasZcl = "Layer ZBEE_ZCL" in cmd;
const isLinkStatus = packet["Packet Type"] === "Link Status (0x08)";

[...]

if (hasZcl) {
  const publishTs = Math.floor(Date.now() / 1000);
  const deviceName = getField(packet, "Device Name ZigBee Source");

  zclMsg = {
    topic: deviceName,
    payload: {
      timestamp: publishTs,
      id: msg.sub_id,
      "wpan.src": getHexAddr(packet, "Source Address"),
      "wpan.dst": getHexAddr(packet, "Destination Address"),
      "zbee.src": getHexAddr(packet, "Source Address ZigBee"),
      "zbee.dst": getHexAddr(packet, "Destination Address ZigBee"),
      topic: `ZigBee/${deviceName}`,
      payload: getField(packet, "Command String")
    }
  };

  // extract target attributes: Active Power, RMS Current, RMS Voltage
  const attrs = asArray(cmd["Attribute"]);
  const statuses = asArray(cmd["Status"]);
  const dataTypes = asArray(cmd["Data Type"]);
  const TARGET_NAMES = { "active power": "Active Power", "rms current": "RMS Current", "rms voltage": "RMS Voltage" };
  const sequence = getField(cmd, "Sequence Number");
  const valueCursors = {};
  let filteredNo = flow.get("filteredNo") || 0;
  // populate message fields
  [...]
}

```

```

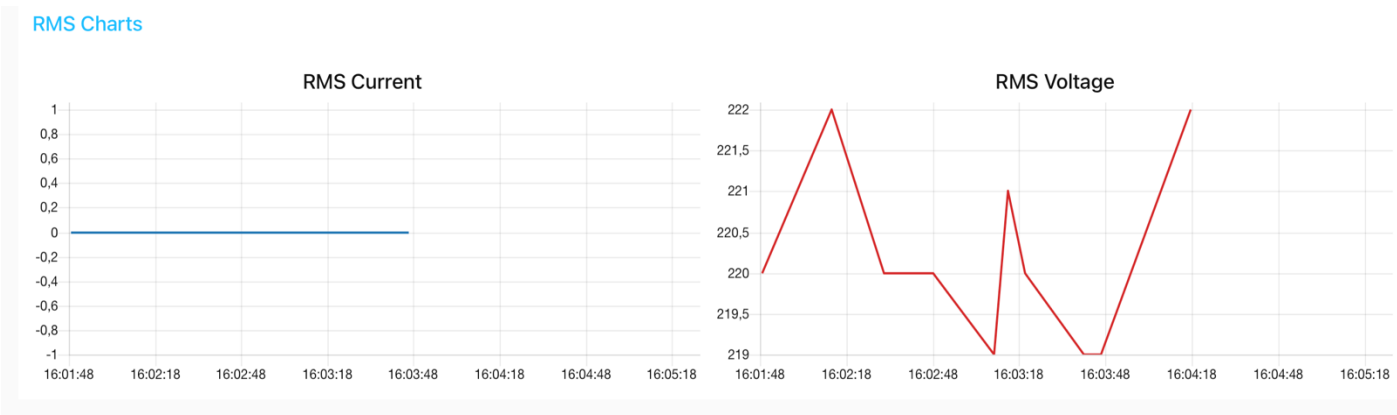
if (isLinkStatus) {
  const outgoing = flow.get("outgoingCostMap") || {};
  const src = getHexAddr(packet, "Source Address ZigBee");
  if (src && cmd["Links"] && Array.isArray(cmd["Links"])) {
    outgoing[src] ||= {};
    cmd["Links"].forEach(link => {
      if (!link || typeof link !== "object") return;
      const dst = getHexAddr(link, "Address");
      const cost = getField(link, "Outgoing Cost");
      if (dst && cost) outgoing[src][dst] = cost;
    });
    flow.set("outgoingCostMap", outgoing);
  }
}

if (msg.sub_count === 200) {
  const outgoing = flow.get("outgoingCostMap") || {};
  const sources = Object.keys(outgoing).sort(hexSort);
  // build outgoing cost messages and add them to the thingspeak queue
  [...]
  endMsg = { payload: `Processed first 200 subscribed IDs. outgoing_cost rows: ${lineNo}. Thingspeak queue:
  ${thingSpeakMsgs.length}.` };
}

return [
  zclMsg, // Output 1: MQTT message
  filteredMsgs.length ? filteredMsgs : null, // Output 2: filtered_elems.csv rows
  currentMsgs.length ? currentMsgs : null, // Output 3: RMS Current chart
  voltageMsgs.length ? voltageMsgs : null, // Output 4: RMS Voltage chart
  outgoingMsgs.length ? outgoingMsgs : null, // Output 5: outgoing_cost.csv rows
  thingSpeakMsgs.length ? thingSpeakMsgs : null, // Output 6: ThingSpeak API payloads
  endMsg // Output 7: End message
];

```


Dashboards



Node-Red RMS charts



Thingspeak channel dashboard